

COMP0141 Security

Table of Contents

- [COMP0141 Security](#)
 - [Table of Contents](#)
 - [Question 1: CIA](#)
 - [I. CIA](#)
 - [II. Threats \(STRIDE\)](#)
 - [III. Security Principles \(ELLF COPS\)](#)
 - [Question 2: Confidentiality & Integrity](#)
 - [I. Encryption](#)
 - [II. Integrity](#)
 - [Question 3: Human Factors](#)
 - [I. Human Limitations](#)
 - [Question 4: Network Security](#)
 - [I. Denial of Service \(DoS\)](#)
 - [1. What is a DoS attack?](#)
 - [2. How are denial of service attacks related to botnets?](#)
 - [3. What techniques can be used to prevent this type of attack?](#)
 - [4. What is a TCP SYN flood?](#)
 - [II. Network Intrusion Detection Systems \(NIDS\)](#)
 - [III. Security Mechanisms](#)
 - [IV. Malware](#)
 - [Question 5: Authentication](#)
 - [I. Authentication Factors](#)
 - [II. Multi-Factor Authentication \(MFA\)](#)
 - [III. Biometrics](#)
 - [Advantages](#)
 - [Disadvantages](#)
 - [IV. Password Salting](#)
 - [V. One-Time Password \(OTP\)](#)
 - [VI. Identification vs Verification](#)
 - [Question 6: Vulnerabilities](#)
 - [I. Security Properties](#)
 - [II. Buffer Overflow](#)
 - [1. What is a buffer overflow?](#)
 - [2. How can a buffer overflow be exploited?](#)
 - [3. Mitigations for Buffer Overflow](#)
 - [4. Why are buffer overflows more common in some languages?](#)
 - [III. SQL Injection](#)
 - [1. What is SQL Injection?](#)
 - [2. Example](#)
 - [3. Consequences for SQL Injection](#)
 - [4. Mitigations for SQL Injection](#)

- IV. Clickjacking
 - V. Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF)
 - VI. Cookies
 - 1. What is a cookie?
 - 2. Two types of cookies
 - 3. What are cookies used for?
 - Question 7: Access Control
 - I. Core Concepts
 - II. Policy Models
 - III. UNIX Permissions
-

Question 1: CIA

I. CIA

- **Confidentiality**: Not disclosed to unauthorized parties.
- **Integrity**: Not improperly altered.
- **Availability**: Available when needed.

II. Threats (STRIDE)

- **Spoofing** -- Authenticity
- **Tampering** -- Integrity
- **Repudiation** -- Non-repudiation
- **Information Disclosure** -- Confidentiality
- **Denial of Service** -- Availability
- **Elevation of Privilege** -- Authorization

III. Security Principles (ELLF COPS)

1. **Economy of Mechanism** -- Keep the design as simple and small as possible.
2. **Fail-Safe Defaults** -- Deny access by default.
3. **Complete Mediation** -- Requires authorization for each protected operation.
4. **Open Design**
 - Public: Encryption Algorithms, Protocols.
 - Private: Cipher Keys, Passwords.
5. **Separation of Privilege** -- Approval from multiple individuals or organizations.
6. **Least Privilege**
 - Use minimum privileges necessary to complete a task.
 - Limit the scope of damage caused by attacks.
7. **Least Common Mechanism** -- Minimize shared mechanisms between users.
8. **Psychological Acceptability** -- Make the security mechanism easy and intuitive to use.

[Back to Table of Contents](#)

Question 2: Confidentiality & Integrity

I. Encryption

1. **Symmetric Encryption:** The sender and receiver **share the same secret key** to encrypt and decrypt messages.
 - Pro: Fast and efficient
 - Con: Key distribution and management is difficult.
2. **Asymmetric Encryption:** Each side has a pair of keys — **public key for encryption** and **private key for decryption**.
 - Pro: Solves the key distribution problem.
 - Con: Slower than symmetric encryption.
3. **Hybrid Encryption:** Use asymmetric encryption to **exchange a symmetric key**, then use symmetric encryption for the actual message.

II. Integrity

1. **Digital Signatures:** Asymmetric, sign with **private key**, verify with **public key**
 - Pro: **Non-repudiation**
 - Con: Computationally expensive
2. **Message Authentication Codes (MACs):** Symmetric, generated with a **shared secret key**
 - Pro: Fast and efficient
 - Con: No non-repudiation.
3. **Hash Functions:** One-way function
 - **Deterministic:** same input -> same output
 - **Pre-image resistant:** output -X-> input
 - **Collision resistant:** hard to find two different inputs with the same output

Authenticated Encryption with Associated Data (AEAD): **Encryption (Confidentiality) + MAC (Integrity/Authenticity)**

[Back to Table of Contents](#)

Question 3: Human Factors

I. Human Limitations

1. **Limited Memory:** users **cannot remember long & complex passwords**, they may **write them down / reuse them**, increasing the risk of compromise.
2. **Limited Attention:** users usually **prioritize their primary task over security**, they may **ignore security warnings / use insecure workarounds**.
3. **Cognitive Bias:** Users may **misjudge the security risks**, e.g.,
 - **Optimism Bias:** "It won't happen to me."
 - **Anchoring Bias:** "I didn't have a problem before, so I'll do it again."
4. **Desensitization:** Too frequent **security warnings / false positives** -> Users **ignore warnings**.

Security Policy should be human-friendly—easy to understand and follow—so that users **comply more and security risks are reduced**.

[Back to Table of Contents](#)

Question 4: Network Security

I. Denial of Service (DoS)

1. What is a DoS attack?

A Denial of Service attack **makes a service unavailable to legitimate users**, usually by **flooding it with requests**. This **exhausts resources** such as **bandwidth/CPU**.

2. How are denial of service attacks related to botnets?

A botnet is a network of **compromised devices** controlled via a **Command and Control Server (C&C Server)**. The attacker can order all bots to flood the same target at once, causing a **Distributed Denial of Service (DDoS)** attack.

3. What techniques can be used to prevent this type of attack?

1. **Client Puzzles**: make clients **perform computational work** before requests are processed, to **slow down attackers**.
2. **CAPTCHAs (Completely Automated Public Turing test to tell Computers and Humans Apart)**: verify requests are from humans, not bots.
 - Pro:
 - Limit bot traffic and spam.
 - Easy to add to a website.
 - Con:
 - Annoying to users.
 - Arms race between CAPTCHA designers and attackers.
3. **Source Identification**:
 - **Ingress Filtering**: ISPs reject packets with **spoofed source IP addresses**, making attack sources easier to identify and block.
 - **Traceback**: Routers log packet paths, allowing the attack sources to be traced back.

4. What is a TCP SYN flood?

An attacker sends **numerous SYN requests** without completing the handshake, causing the server to **allocate resources for half-open connections**, which exhausts the server's resources and prevents legitimate users from connecting.

Mitigation: SYN Cookies: Avoid allocating resources until the handshake is completed.

II. Network Intrusion Detection Systems (NIDS)

1. Signature-Based Detection:

- Detects **known** attacks by comparing network traffic to **a database of attack signatures**.
- May miss new attacks.

2. Anomaly-Based Detection:

- Detects **unknown** attacks by comparing network traffic to **a baseline of normal behavior**.
- May produce false positives.

III. Security Mechanisms

1. **Firewalls**: Filter network traffic and prevent **unauthorized network access to or from a private network** based on **predefined security rules** (IP addresses, ports, and protocols).
2. **Network Intrusion Detection Systems (NIDS)**: Monitor network traffic for **suspicious activity** and **alert administrators** of potential security risks.
3. **Virtual Private Networks (VPNs)**: create an **encrypted tunnel** over an untrusted network, **protecting data from eavesdropping and modification in transit**. The user must **authenticate before the tunnel opens**.
4. **Hypertext Transfer Protocol Secure (HTTPS)**: Protect data from eavesdropping by encrypting network traffic; it does not hide IP addresses.

IV. Malware

	Need a Host Program	Self-Contained Program
Self-Spreading	Virus	Worm
Non-Spreading	Trojan Horse, Rootkit	Spyware, Keylogger, Dialer

- Virus: A program that infects other files by inserting a copy of itself; it **cannot survive alone** and runs **when the infected file is executed**.
 - Where it hides: File infection (overwrite / parasitic), macro infection (auto-run macros in Word/Excel/PDF)
 - Defense:
 1. **Signature-based detection**: match against a database of known-malware byte/instruction patterns
 2. **Heuristics** (signs of infection),
 3. **Behavioral signatures**: match against a baseline of normal program behavior,
 4. **Sandboxing**: run untrusted applications in a restricted environment.
- Worm: A **self-spreading, self-contained** program that **autonomously spreads over a network** by sending copies of itself to other nodes — no host file needed.
 - Propagation: email harvesting, network-share enumeration, IP scanning; exploit-based worms need **no human interaction**.
 - Defense:
 1. **Virus scanners** — effective against email-based worms.
 2. **Host-level**: patching, **stack protection**, **ASLR** (Address Space Layout Randomization).
 3. **Network-level**: **IDS**, **limit outgoing connections**, **personal firewall** (block outgoing SMTP from unknown apps).
- Trojan Horse: Acts as a **backdoor entry** into the system, often while **pretending to be a legitimate program**.
- Ransomware: Encrypts files and demands payment for the decryption key.

[Back to Table of Contents](#)

Question 5: Authentication

I. Authentication Factors

1. Something You **Know**:
 - Passwords
 - PINs (Personal Identification Numbers)
 - Personal Details / Security Questions
2. Something You **Have**:
 - Smart Cards
 - Authentication Apps
3. Something You **Are** (Biometric):
 - Fingerprint
 - Facial Recognition

Challenge-response: the server sends a **random challenge**, and the client **returns a response computed with its secret key**. This is a sequential process that cannot be performed concurrently, and it **prevents replay attacks**.

II. Multi-Factor Authentication (MFA)

- Different factors have **different weaknesses**, one attack cannot break both.
- **Compromising one factor is not enough; an attacker must break multiple factors, which requires greater cost and effort.**

III. Biometrics

Should be **Distinguishable & Repeatable**.

Sample -> Template (Feature Extraction) -> Matching

Advantages

- **Nothing to remember**
- **Cannot share**
- **Unique** (Assuming perfect accuracy)

Disadvantages

1. **Matching is never perfect**
 - There is a trade-off between the **False Acceptance Rate (FAR)** and the **False Rejection Rate (FRR)**.
 - A stricter threshold lowers FAR but raises FRR: more security, but legitimate users are falsely rejected more often.
2. **Cannot be revoked**
 - Biometrics are private but not secret.
 - We may leave them on glasses or door handles.
 - A stolen fingerprint cannot be revoked like a stolen password. The leak is permanent.
3. **Can be faked**

- The sensor only sees what is presented, so a photo or a fake finger may pass.

Users may refuse to provide biometric data because:

- Threats to privacy.
- Doubts about the reliability and security of biometric systems.
- Fear of being sold for commercial purposes.

IV. Password Salting

A random value added to a password before hashing.

- Prevents **rainbow table attacks**.
- Prevents **identical passwords having identical hashes**.

A password should be stored as `hash(password, salt)`, and the salt should be **stored in plaintext** alongside the hash.

V. One-Time Password (OTP)

1. SMS (Short Message Service) OTP

- **Pro:** No extra app needs to be installed.
- **Pro:** Setup is simple, only requires a phone number.
- **Con:** Vulnerable to **SIM swapping / SMS interception**.

2. Authentication App

- **Pro:** Works without a network connection.
- **Pro:** Not vulnerable to **SIM swapping / SMS interception**.
- **Con:** Setup is more complicated.

3. Hardware Token

VI. Identification vs Verification

- Identification: determining **Who are you?** by asking for a username or ID.
- Verification: determining **Are you really who you say you are?** by validating credentials (password, OTP, biometric).

[Back to Table of Contents](#)

Question 6: Vulnerabilities

I. Security Properties

1. **Correctness:** input ----> correct output
2. **Safety:** input --X--> dangerous output
3. **Robustness:** handle errors that may occur during execution.

These properties must hold even in the presence of **resourceful and strategic adversaries**.

II. Buffer Overflow

```
argument1
argument2
argument3
Return Address
-----
Saved fp
local variable1
local variable2
```

1. What is a buffer overflow?

A buffer overflow occurs when **the data a program writes goes beyond the buffer's boundary and overwrites adjacent memory.**

2. How can a buffer overflow be exploited?

The overflow may overwrite:

1. Local Variables:

- Data Tampering;
- Program logic errors;
- Authentication bypass.

2. Return Address

3. Saved Frame Pointer

- Control-flow Hijacking
- Execution jumps to an attacker-controlled address and runs *shellcode* with a NOP sequence. The attacker gains arbitrary code execution with the program's privileges.

3. Mitigations for Buffer Overflow

1. **Bounds Checking:** Input Length < Buffer Size
2. **Use Safer Libraries/APIs.** e.g. `strncpy()` instead of `strcpy()`, `strncat()` instead of `strcat()`, etc.
3. **Stack Canaries**
4. **Non-executable Stack**

4. Why are buffer overflows more common in some languages?

- **C/C++** allow **direct memory access & do not automatically check array bounds.**
- Memory-safe languages such as **Java/Python** perform **bounds checking & memory management.**

III. SQL Injection

1. What is SQL Injection?

SQL Injection occurs when **user input includes SQL syntax and is treated as part of SQL commands instead of data**, and the system **executes the attacker's SQL code without validation.**

2. Example


```
SELECT * FROM users
WHERE username = 'USER'
AND password = 'PASSWORD';
```

The attacker enters:

```
' OR '1'='1
```

The query becomes:

```
SELECT * FROM users
WHERE username = '' OR '1'='1'
```

The condition `'1'='1'` is always true, so the attacker can bypass authentication.

3. Consequences for SQL Injection

1. Bypass authentication.
2. Read confidential data. (Confidentiality)
3. Modify or delete database records. (Integrity)
4. Disrupt service. (Availability)

4. Mitigations for SQL Injection

1. Use **prepared statements** (with parameterised queries) to ensure user input is treated as data, not code.
2. **Server-side Input Sanitization**: Accept only safe input or correctly escape input.

IV. Clickjacking

The attacker puts an **invisible layer** over a normal-looking page, tricking a user into clicking a **hidden or disguised button** on a trusted website, causing an unintended action.

- Mitigation: **X-Frame-Options** or **CSP frame-ancestors** — stop the site being loaded inside a frame.

V. Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF)

1. **Cross-Site Scripting (XSS): Injecting malicious scripts** into a trusted website, so the victim's browser **runs it with that site's authority**, and the script can read cookies and session tokens.
 - Mitigation: **Content Security Policy (CSP) / Input Sanitization (allowlist)**
2. **Cross-Site Request Forgery (CSRF)**: Tricking a logged-in user's browser into **sending unauthorized requests** with the user's cookies, so the site accepts the request as legitimate.
 - Mitigation: **CSRF token** (a random string attached to session that is not sent automatically) / **SameSite cookie**.

- **XSS** exploits the **client's trust in the server** (the browser trusts the site, so it runs the script).
- **CSRF** exploits the **server's trust in the client** (the server trusts the request because it carries the right cookie).

Examples of an attacker's actions:

- change the victim's email address or password
- transfer money from the victim's bank account
- take over the victim's account
- stored XSS: **stored by the server** and later delivered to the victim's browser (e.g., in a forum post)
- reflected XSS: included **in a server's response** after the victim clicks a malicious link (e.g., in an email)

VI. Cookies

1. What is a cookie?

A cookie is a **name–value pair** stored by the browser and **automatically attached to requests**.

2. Two types of cookies

1. **Session cookies**: deleted when the browsing session ends (the browser is closed).
2. **Persistent cookies**: stay until a fixed expiry date

3. What are cookies used for?

1. **Session management**: keep the user logged in, remember a shopping cart.
2. **Personalisation**: remember user preferences.
3. **Tracking**: record what the user viewed, so the site can show targeted ads.

(Mnemonic: **S**ession / **P**ersonalisation / **T**racking — "SPT")

[Back to Table of Contents](#)

Question 7: Access Control

I. Core Concepts

- Access Control permits or denies a **subject** access to an **object** with a specific right (read/write/execute).

What are the different types of access control mechanisms?

1. Access control lists,
2. Access control matrices,
3. Capability/attribute based,
4. Role based.

- Give only the permissions required by business functions (Least Privilege) and deny everything else by default (Fail-safe Default). Complete Mediation ideally checks every access, but this may create excessive performance overhead.
- Access control limits **both external and insider threats**, but can be hard to update, scale, and implement without policy errors.

II. Policy Models

- **DAC (Discretionary Access Control)**: the object owner controls and may pass on permissions.
- **MAC (Mandatory Access Control)**: permissions are assigned by a central administrator or system policy.
- **RBAC (Role-Based Access Control)**: permissions are assigned to roles instead of individuals

III. UNIX Permissions

- Permissions use **type | owner | group | others**, where **r** = read, **w** = write, and **x** = execute.
- Check only one set: owner->group->others
- For a directory, r lists contents, w creates/deletes/renames entries, and x allows traversal.
- A sticky bit (t) means only the file owner, directory owner, or root can delete/rename a file in the directory.
- Owner permissions override group permissions, so an owner may be denied access even if their group has it (negative permission).